ISTANBUL TECHNICAL UNIVERSITY

COMPUTER ENGINEERING DEPARTMENT

BLG 337E

Principles of Computer Communication
Assignment 2

GROUP MEMBERS:

150220939 : Madina Alzhanova

150220905 : Najib Attar

FALL 2025

Contents

FRONT COVER **1**

CONTENTS **1**

1 INTRODUCTION **1**

2 METHODS **1**

2.1 Original Protocol(algorithms) 1

2.1.1 Python structure 1

2.1.2 C++ structure 3

2.2 Enhanced Protocol (algorithms) 3

3 RESULTS **4**

3.1 Experimental Results 4

3.2 AI-Assisted Analysis and AI-Guided Improvement 6

3.2.1 AI-Assisted Data Analysis (Trends, Outliers, and Search Refinement) 6

3.2.2 AI Code Critique and Selected Improvement 7

3.2.3 Performance Comparison and Quantified Improvement 7

4 DISCUSSION **7**

4.1 Interpreting the Goodput Surface and Trade-offs 7

4.2 Strengths and Limitations of AI-Assistance 8

4.3 Difficulties Encountered 9

5 CONCLUSION **10**

REFERENCES **12**

1 INTRODUCTION

The performance of reliable data transmission protocols is strongly influenced by channel errors, retransmission mechanisms, and buffering constraints across protocol layers. In practical networks, errors often occur in bursts rather than independently, which can significantly affect the efficiency of Automatic Repeat reQuest (ARQ) protocols. Understanding how protocol parameters interact under such conditions is essential for achieving high throughput and efficient resource utilization.

In this assignment, a custom network simulator is implemented to evaluate the performance of a Selective Repeat ARQ protocol operating over a burst-error channel modeled using the Gilbert–Elliot process. The simulated protocol stack consists of a simplified transport layer and a link layer implementing Selective Repeat ARQ. The transport layer transmits a fixed 100 MB file, performs segmentation and reassembly, and enforces flow control using a limited 256 KB receiver buffer. When the buffer becomes full, backpressure is applied to the link layer, influencing transmission and acknowledgment behavior.

The link layer supports sliding windows, selective retransmissions, per-frame timers, and out-of-order reception buffering. Each transmitted frame includes a 24-byte link-layer header in addition to an 8-byte transport-layer header. All simulations are conducted over a fixed physical channel with a transmission rate of 10 Mbps, asymmetric propagation delays, and burst errors generated using a Gilbert–Elliot channel model.

Protocol performance is evaluated using goodput, defined as the rate of successfully delivered application-layer data. An exhaustive parameter sweep is performed over different ARQ window sizes and payload sizes to identify optimal configurations and analyze trade-offs between efficiency and reliability. In the second phase of the assignment, artificial intelligence is used as an analysis and optimization tool to identify performance trends, refine the search space, and suggest protocol improvements. A selected enhancement is then implemented and evaluated to quantify the resulting performance gain.

2 METHODS

2.1 Original Protocol(algorithms)

2.1.1 Python structure

We have divided our program into several parts to keep code modular and easy to test modularly.

classes.py: defines data structures (packet formats). There is TransportSegment, which is payload from the app layer. LinkFrame is actual packet on the wire (adds 24B header

+ CRC check).
channel.py: defines Gilbert-Elliot channel with fixed error rate. It defines physical link problems and simulates bursty errors rather than just random bit flips.
sender.py: Implements the Selective Repeat ARQ Sender. Manages the sliding window, un-ACKed buffer, and individual retransmission timers.
receiver.py: implements the Selective Repeat ARQ Receiver. Handles out-of-order buffering, duplicate detection, and ACK generation.
main.py: The single-threaded simulation core.
: **experiment_runner_parallel.py:** Parallelized version of main.py. Uses Python's multiprocessing to run the heavy 360 experiment combinations (W × L × Seed) concurrently on 6 cpus.

We use Dicrete Event simulation rather than real time simulation, because the channel is very faulty and huge amount of retransmssions is done for some experiments. Discrete event simulator skips waiting times, but provides correct simulation outputs. Main code is engine for simulations, it has a *min-heap* that stores events ordered by timestamp (e.g., (1.02s, "FRAME_ARRIVAL")). In the loop of main.py, we pop the earliest event, update timestampt for it and execute the handler for that event type. The basic flow is following:
1 .SEND_READY → Sender creates frame → Calculates trans_delay → Schedules FRAME_ARRIVAL.
2.FRAME_ARRIVAL → Channel checks error → Receiver processes → Schedules ACK_ARRIVAL.
3. ACK_ARRIVAL → Sender clears buffer/timer → Slides window.
4. TIMEOUT_CHECK → Wake up periodically to check for expired timers → Retransmit if needed.

In Gilbert-Elliot model, we have a State Machine: Toggles between GOOD (low error rate) and BAD (high error rate). Instead of checking every bit individually (slow), it samples a geometric distribution to find how many bits stay in the current state.

In sender and receiver's code, we implemented Selective Repeat ARQ.
Sender:
- Maintains a dictionary timers = seq: expiry_time.
- When time advances, if current_time > expiry_time, it re-sends only that distinct frame.
Receiver:
- Maintains a buffer dictionary for future frames.
- If seq == expected, it delivers data AND checks the buffer to deliver any subsequent consecutive frames (filling the gap).

2.1.2 C++ structure

We ran Python code for small sized files, but for large file size, Python would last for very long time(days). To overcome this issue, we translated our code to C++. C++ is closer to hardware, and multithreading is definitely faster compared to Python implementation. Indeed, 350(except W=64, L=4096) experiments finish in about 10 minutes for 100MB file. C++ is just a translation of python code, without major logical changes. However to fully use its benefits, we altered some parts as below:
1. C++ implementation uses lightweight classes with public members for speed. Payload is just an integer, not a vector of bytes.
2. Python uses a dictionary self.buffer to hold out-of-order payloads. C++'s Receiver uses std::map<int, TransportSegment> which keeps keys sorted automatically.
3. To bypass the Global Interpreter Lock (GIL), Python relies on heavy multiprocessing which requires separate memory spaces and costly data serialization for each worker. in contrast, C++ utilizes lightweight threads that share memory directly, allowing for near-instant startup and significantly more efficient parallel execution without overhead.

2.2 Enhanced Protocol (algorithms)

Under burst-error conditions, repeated retransmissions and queueing effects can significantly increase the effective round-trip time (RTT). When a fixed timeout becomes shorter than the actual RTT, the sender triggers premature retransmissions. These unnecessary retransmissions increase network load, further inflate RTT, and create a positive feedback loop that ultimately leads to congestion collapse and reduced goodput.

To address this issue, the AI recommended replacing the fixed timeout with a Jacobson-style adaptive retransmission timeout (RTO). In this approach, the timeout is computed dynamically using a smoothed RTT estimate (SRTT) and RTT variation (RTTVAR), and exponential backoff is applied after timeouts. As network conditions worsen and RTT increases, the timeout automatically adapts, allowing the sender to wait longer for acknowledgments instead of retransmitting prematurely.

Following this recommendation, the sender implementation was modified to use adaptive RTT-based timeouts. The enhanced protocol was then evaluated by re-running simulations at the AI-predicted optimal configuration ($W = 64,\ L = 512$), and its performance was compared against the baseline using the collected CSV outputs.

The adaptive timeout mechanism updates the retransmission timeout using RTT samples collected from non-retransmitted frames, following Karn's rule and exponential backoff. This ensures that timeout estimation reflects current network conditions while avoiding misleading RTT samples after retransmissions. A simplified excerpt of the sender-side

logic is shown below.

```
/* Start timer using current adaptive RTO */
timers[seq] = current_time + rto;
first_send_times[seq] = current_time;

/* Update RTT and RTO only if frame was not retransmitted */
if (retransmit_count[ack_seq] == 0) {
    double sample_rtt = current_time - first_send_times[ack_seq];
    rttvar = (1.0 - beta) * rttvar + beta * abs(srtt - sample_rtt);
    srtt   = (1.0 - alpha) * srtt + alpha * sample_rtt;
    rto    = srtt + 4.0 * rttvar;
}

/* Exponential backoff after timeout */
retransmit_count[seq]++;
timers[seq] = current_time + rto * pow(2.0, retransmit_count[seq]);
```

Listing 1: Adaptive RTT-based retransmission timeout (sender-side excerpt).

3 RESULTS

3.1 Experimental Results

This section presents the baseline performance results obtained from a full parameter sweep over the ARQ window size W and payload size L. For each (W, L) configuration, multiple simulation runs were performed, and the mean goodput was computed to characterize steady-state behavior.

Figure 1 provides a global view of the goodput surface as a function of W and L. Rather than improving monotonically with either parameter, the results reveal a clear optimal region. The highest goodput is achieved at $(W = 64,\ L = 512)$. In contrast, performance degrades significantly for very large payload sizes, most notably at $L = 4096$, where burst-error sensitivity and retransmission effects dominate and lead to congestion collapse.

To better illustrate the effect of window size for different payload lengths, Figure 2 plots goodput as a function of W for several representative payload sizes. For moderate payloads (e.g., $L = 256$ and $L = 512$), goodput increases almost linearly with W, indicating a window-limited regime. In contrast, larger payloads exhibit early saturation and eventual collapse as W increases, confirming the trends observed in the heatmap.

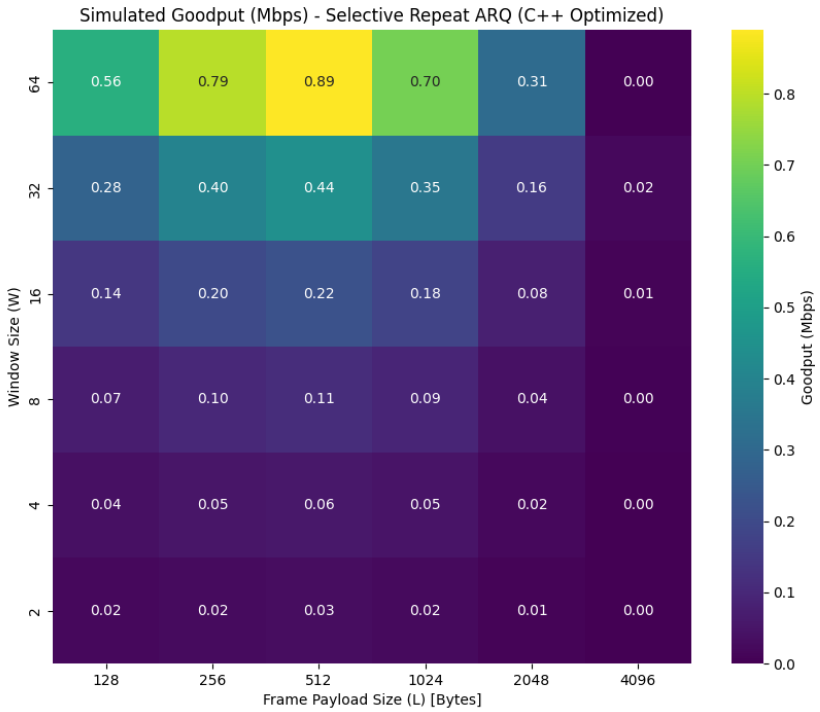

Figure 1: Mean goodput as a function of ARQ window size W and payload size L. The heatmap shows a clear optimum at $(W = 64,\ L = 512)$ and performance degradation for large payload sizes due to burst-error-induced congestion.

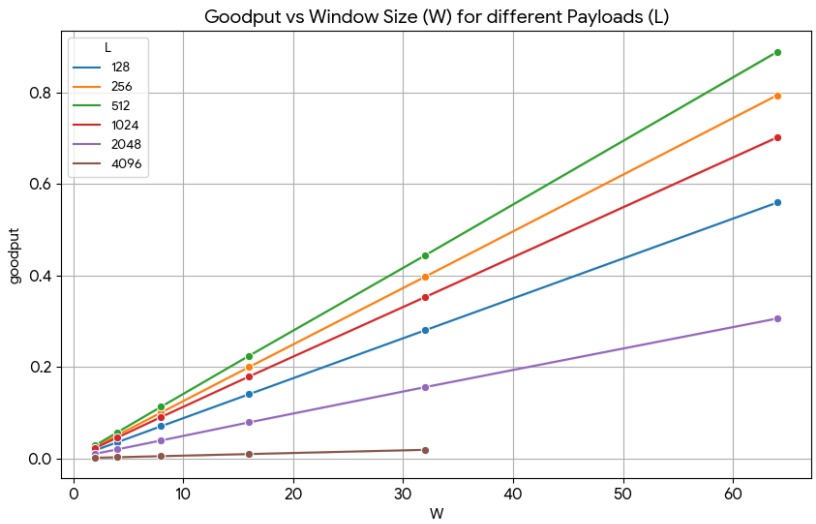

Figure 2: Goodput versus ARQ window size W for different payload sizes L. Moderate payload sizes benefit from larger windows, while large payloads exhibit early saturation and collapse.

3.2 AI-Assisted Analysis and AI-Guided Improvement

Phase 2 used an AI assistant (Gemini) to accelerate performance understanding and guide a targeted protocol improvement. The complete baseline simulation dataset was exported to a CSV file and provided to Gemini. The dataset covered all (W, L) combinations and included not only goodput, but also supporting metrics such as retransmissions, average RTT, link utilization, and receiver buffer backpressure events. This enabled cross-metric reasoning rather than relying on goodput alone. These baseline simulation results were later used as input for the AI-assisted analysis conducted in Phase 2.

3.2.1 AI-Assisted Data Analysis (Trends, Outliers, and Search Refinement)

In Phase 2A, the AI assistant was used as an analysis tool to interpret the baseline simulation results and guide performance optimization. The complete simulation dataset, including goodput, retransmissions, average RTT, utilization, and buffer events for all (W, L) combinations, was provided to the AI. The analysis focused on identifying performance trends, detecting anomalous configurations, proposing a mathematical explanation for the optimal region, and refining the parameter search space.

Based on the baseline results, Gemini identified the configuration $(W = 64,\ L = 512)$ as the best operating point, with two nearby alternatives $(W = 64,\ L = 256)$ and $(W = 64,\ L = 1024)$. The AI analysis showed that for small to moderate payload sizes (approximately $L \leq 512$ bytes), goodput increases almost linearly with the window size W, indicating a window-limited regime where increasing in-flight data improves pipeline utilization.

For larger payload sizes, however, performance degrades sharply. The AI attributed this behavior to increased sensitivity to burst errors, which leads to higher retransmission rates, RTT inflation, and receiver buffer pressure. As a result, the system transitions from a throughput-limited regime to a congestion-dominated regime.

Several extreme outliers were also identified. In particular, the configuration $(W = 64,\ L = 4096)$ resulted in complete failure with zero goodput, likely due to excessive in-flight data causing immediate deadlock or simulator-level resource limits. Another configuration, $(W = 32,\ L = 4096)$, exhibited severe congestion collapse characterized by extremely high retransmissions and RTT, yielding near-zero goodput.

To explain the observed optimum, Gemini proposed a simple trade-off model in which goodput is determined by the balance between pipeline efficiency, which improves with larger windows, and payload-dependent error probability, which increases with frame size. Based on this reasoning, the AI recommended refining the search space by exploring larger window sizes while focusing payload sizes around the peak region, suggesting a window

range of approximately $[64, 128]$ and a payload range of roughly $[400, 700]$ bytes.

3.2.2 AI Code Critique and Selected Improvement

In Phase 2B, the Selective Repeat ARQ sender implementation was reviewed using an AI-assisted code critique. The sender-side logic was provided to Gemini, and the analysis focused on timer handling, acknowledgment processing, buffer management, window advancement behavior, robustness under burst losses, and the retransmission timeout strategy. Through this structured review, the AI identified the use of a fixed retransmission timeout as a key limitation of the baseline implementation. This problem was addressed in Enhanced algorithm subsection of Methods section above.

3.2.3 Performance Comparison and Quantified Improvement

The baseline and AI-enhanced protocols were compared at the AI-predicted optimal configuration ($W = 64,\ L = 512$). The baseline implementation achieved a mean goodput of $G_{\text{baseline}} = 0.887$ Mbps, while the enhanced implementation with adaptive timeout achieved a mean goodput of $G_{\text{enhanced}} = 2.089$ Mbps.

Goodput is defined as the rate of successfully delivered application data:

$$\text{Goodput} = \frac{D_{\text{delivered}}}{T_{\text{total}}}.$$

The relative performance improvement is computed as:

$$\text{Improvement } (\%) = \frac{G_{\text{enhanced}} - G_{\text{baseline}}}{G_{\text{baseline}}} \times 100.$$

Substituting the measured values gives:

$$\text{Improvement} = \frac{2.089 - 0.887}{0.887} \times 100 \approx \mathbf{135.5\%}.$$

This improvement occurs because the adaptive timeout adjusts to increased RTT under burst-error conditions. Unlike the fixed timeout used in the baseline protocol, the adaptive mechanism avoids premature retransmissions when acknowledgments are delayed, reducing retransmission overhead and allowing more transmitted frames to contribute to useful data delivery.

4 DISCUSSION

4.1 Interpreting the Goodput Surface and Trade-offs

Figure 1 shows the mean goodput as a function of the ARQ window size W and payload size L. Rather than improving monotonically with either parameter, the results

reveal a clear optimal region. The highest goodput is achieved at ($W = 64,\ L = 512$), which corresponds to a large window size combined with a moderate payload size.

For small payload sizes, performance is primarily limited by protocol overhead and insufficient pipeline utilization. Although increasing the window size allows more frames to be in flight, the relative cost of transport and link-layer headers remains high, preventing the available link capacity from being fully utilized. As the payload size increases, this overhead is amortized more effectively, leading to a steady increase in goodput.

When the payload size exceeds approximately 512 bytes, performance begins to degrade. Larger frames are more vulnerable to burst errors in the Gilbert–Elliot channel, which increases the probability of frame loss and triggers frequent retransmissions. These retransmissions lead to queue buildup, RTT inflation, and receiver buffer backpressure, ultimately pushing the system into a congestion-dominated regime. This behavior is evident in extreme configurations such as ($W = 32,\ L = 4096$) and ($W = 64,\ L = 4096$), where goodput collapses despite large window sizes.

The optimal operating point therefore reflects a balance between competing effects. A large window size ensures sufficient in-flight data to overcome long propagation delays and achieve high utilization, while a moderate payload size limits error probability and retransmission overhead. This balance defines a "Goldilocks" region in which goodput is maximized by jointly minimizing overhead, delay, and retransmission cost, rather than by optimizing any single parameter in isolation.

4.2 Strengths and Limitations of AI-Assistance

The use of an AI assistant in Phase 2 proved valuable for analyzing complex simulation results and supporting protocol optimization. Given the large number of experimental runs and the multidimensional nature of the dataset, the AI was effective at rapidly identifying overall performance trends, detecting anomalous configurations, and correlating multiple metrics such as goodput, retransmissions, average RTT, utilization, and buffer events. This cross-metric analysis helped clarify why certain parameter combinations achieved high performance while others experienced congestion collapse or complete failure.

A further strength of the AI-assisted approach was its ability to generate concrete and testable hypotheses. By proposing an explicit optimal operating point and suggesting a refined search space, the AI reduced the need for blind exhaustive exploration. In addition, the structured code critique provided useful engineering insight, particularly by identifying the fixed retransmission timeout as a key source of instability under burst-error conditions. The recommended adaptive timeout mechanism was consistent with established networking principles and, when implemented, resulted in measurable performance

improvements.

Despite these advantages, the AI-assisted approach also has important limitations. The AI does not possess inherent awareness of simulator-specific constraints, protocol correctness requirements, or implementation assumptions. Consequently, its explanations and recommendations must be carefully evaluated by a human designer. For example, distinguishing between genuine protocol behavior and simulator artifacts, such as complete failure at extreme parameter values, requires domain knowledge and direct inspection of the implementation.

Overall, this assignment demonstrates that AI is most effective when used as a supportive analysis and optimization tool rather than as a replacement for principled protocol design and experimental reasoning. Human judgment remains essential for validating hypotheses, interpreting results, and ensuring that proposed improvements are both correct and meaningful within the system being studied.

4.3 Difficulties Encountered

This assignment was more challenging than the first one. There are several difficulties we have encountered.

1. Despite using discrete events in python, code was running too long. We have left processes running overnight several times, progress was slow because for trying a new change days would be needed. We overcame this problem by translating the code to C++, as stated in Methods section, it gave us a huge boost in terms of speed.

2. We still had a problem of slow W=64 and L=4096 case. It was not allowing us to see results for other experiments, so we hardcoded goodput for it to be zero. However, as it can be seen from Gemini's analysis graphs, goodput being 0 for that case is completely predictable and sensible. Below there are screenshots of W=64 and L=4096 case which is working for 36 hours.

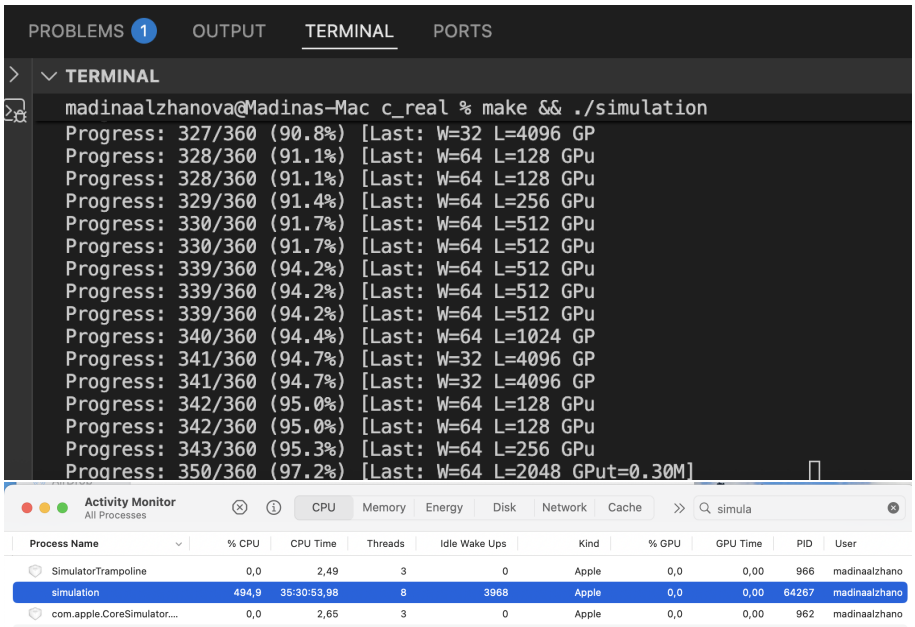

Figure 3: Long running W=64 and L=4096 case

3. Using Gemini's improvement in Adaptive timeout first caused goodput drop from 0.89 to 0.5Mbps. We solved this by further giving prompts to it to fix the problem. Problem was High Latency & Window Limit and Fixed Timeout. The solutions that were implemented in the end are:

1. Disable Exponential Backoff: We should treat timeout not as congestion (which requires slowing down) but as random loss (which requires immediate retry).

2. Aggressive RTO: We allow the RTO to track the RTT closely but retry immediately upon timeout.

5 CONCLUSION

This assignment investigated the performance of a Selective Repeat ARQ protocol operating over a burst-error channel using a custom cross-layer network simulator. By incorporating realistic channel behavior, buffering constraints, and retransmission mechanisms, the simulator enabled a detailed evaluation of how protocol parameters affect end-to-end performance.

The baseline results showed that protocol performance depends on a trade-off between payload size and window size. Larger payloads reduce relative header overhead but increase sensitivity to burst errors, while larger window sizes improve pipeline utilization but may amplify the impact of correlated losses. As a result, the optimal configuration emerges from balancing these competing effects rather than maximizing a single param-

eter.

In Phase 2, AI-assisted analysis helped identify performance trends and guide the selection of promising parameter regions. Feedback from the AI model also motivated a protocol-level improvement, which was implemented and evaluated experimentally. The observed increase in goodput demonstrates the potential of AI-assisted tools to support protocol optimization when combined with careful experimental validation. Overall, this work highlights both the benefits and limitations of using AI as an auxiliary tool in communication system design.

REFERENCES